

Arquitectura y límites

Control

Versión 2.0 | Corte documental: 29/09/2026. Documento técnico para revisión académica. Implementado significa observado en código; no equivale a validación productiva. Las iteraciones son planes de trabajo, no actas ejecutadas.

Componente

Ruta	Tecnología	Responsabilidad	Límite
frontend/web	React + Vite	Landing, acceso, paneles y demo	No autoriza operaciones reales
backend/api	Laravel	Identidad, permisos, estados y transacciones	Fuente de verdad del servicio
python/api	FastAPI	Ranking numérico	No opera cuentas ni solicitudes
database + migraciones	PostgreSQL	Persistencia e integridad	No sustituye validación de API

Recorrido de datos

Navegador -> web/proxy -> Laravel -> PostgreSQL. Laravel -> FastAPI /match, con fallback PHP si falla. Worker Laravel procesa plazos. JS calcula ranking solo para demo localStorage. En producción, API y matching deben quedar detrás de los controles de red previstos.

Decisión de stack

Se mantiene la arquitectura académica del monorepo por separación de responsabilidades y trabajo previo. A30 ilustra un caso Laravel/MySQL, pero no demuestra que MySQL sea obligatorio ni superior. A29 usa otro stack: se comparan capacidades, no marcas. Cambiar motor requiere ADR, migraciones y evaluación propias.

Riesgo de redundancia

La regla se replica en tres lenguajes por finalidad demostrativa. domain/rules.json y fixture compartido evitan cambios manuales divergentes. Laravel verifica ranking remoto contra su cálculo; esto añade trabajo duplicado que debe incluirse en las mediciones.

Diseño técnico: React, Laravel, FastAPI y PostgreSQL

Secuencias técnicas principales

Registro e inicio

React POST /api/register -> validación client/technician -> usuario y perfil en transacción -> 201 sin token. Login verifica hash y acceso -> crea token aleatorio, persiste hash y expiración -> actor. React consulta /state; recién entonces habilita panel. Logout elimina token; 401 borra sesión local.

Creación y matching

Cliente POST actions/create -> permisos -> validación -> transacción de solicitud -> filtrar candidatos -> llamada Python -> comprobar versión, orden y score -> fallback si corresponde -> insertar hasta tres ofertas -> persistir eventos -> devolver ok -> React refresca estado.

Aceptación y confirmación

Técnico POST accept -> lock solicitud -> validar oferta y reloj -> lock técnico -> verificar disponibilidad -> marcar oferta accepted, cancelar otras y fijar agreed_fee -> proposed. Cliente propietario POST confirm -> confirmed. Técnico asignado puede iniciar. Probar dos aceptaciones concurrentes con conexiones diferentes en PostgreSQL.

Finalización y valoración

complete solo asignado y desde in_progress -> completed + completed_at + rating_until; libera disponibilidad. rate solo propietario antes del límite y una vez -> actualiza promedio/reviews con bloqueo + rated. El backend rechaza transición repetida aunque UI se manipule.

Consistencia pendiente

create no incluye actualmente clave de idempotencia; se documenta como brecha de reintentos. El esquema no añade CHECK a todos los catálogos. Antes del lanzamiento se exige prueba concurrente y revisión de orden de bloqueos, no solo suite SQLite.

Diseño técnico: React, Laravel, FastAPI y PostgreSQL

Decisiones arquitectónicas

Registro ADR

Decisión	Motivo	Consecuencia
ADR-01 Mantener políglota	Convención académica y motor independiente	Más contratos y pruebas de paridad
ADR-02 PostgreSQL referencia	Stack y Compose existentes	SQLite local no cierra pruebas de concurrencia
ADR-03 Matching determinista	Trazabilidad y falta de dataset local	Calibrar pesos; no afirmar optimalidad
ADR-04 Aislamiento por propiedad	Marketplace actual, tres roles	No multitenencia implícita
ADR-05 Asistencia opcional	A22 advierte sobre proveedores inventados	LLM no ejecuta decisiones ni sustituye catálogo
ADR-06 Pagos desacoplados	Modelo comercial aún no aprobado	Núcleo funciona sin checkout ni suscripción

Observabilidad propuesta

Medir latencia del ranking, uso de fallback, número de ofertas, vencimientos y errores por acción. Usar IDs de correlación; no guardar contraseñas, tokens, direcciones ni texto libre sensible en logs. Las métricas de producto son distintas de latencia HTTP.

Gobernanza del cambio

Cambio -> ADR -> historia -> prueba -> implementación -> QA -> evidencia -> aceptación. Un artículo inspira una hipótesis; el Product Owner decide su incorporación y el equipo la valida. Preservar versiones de reglas y migraciones.

Riesgos y responsabilidades de seguridad

Riesgo

Riesgo	Control	Prueba
Acceso ajeno	Servidor comprueba rol, propiedad y estado	Pruebas 403 y respuesta sin dirección
Suplantación de habilitación	verify solo admin; no mass assignment	Enviar verified/role en formularios públicos
Sesión suspendida	Revocar tokens y comprobar is_active	Token antiguo no vuelve a servir
Competencia por oferta	Bloqueos y restricciones	Dos clientes DB aceptan al mismo tiempo
Inyección y contenido de anuncio	Validación, escape React, URL restringida	XSS, MIME incorrecto, URL javascript
Caída del motor	Fallback y reglas versionadas	Timeout, score inválido y versión distinta
Sesgo de reputación	Medición, transparencia y revisión propuesta	Comparar exposición de perfiles nuevos

Alcance de protección

No se declara seguridad completa ni cumplimiento certificado. El control de verificación profesional todavía necesita criterios operativos y evidencia. A12, A13 y A18 fundamentan la necesidad de gobernanza; no reemplazan un análisis de amenazas del sistema concreto.

Fuentes

Las referencias [Axx] remiten a las fichas del documento 01 y al catálogo trazabilidad/v2/articulos.csv. Páginas citadas: numeración física del PDF, salvo indicación expresa. Los originales se conservan en fuentes/articulos/Axx.pdf; SHA-256 en fuentes/articulos/manifest.json. Código: Permissions.php, ApiToken.php, Marketplace.php, Matching.php y python/api/app.py.